

Lab 3: Component Modelling & Architectural Pattern Selection

Software Engineering Laboratory

Name	B V Tejas
SRN	PES1UG24CS108
Section	B
Problem Statement (Lab 1)	Household Carbon Footprint & Target Tracker

1. System Overview

The system lets a household record its electricity, gas and transport consumption, converts those readings into a monthly CO₂e total using standard emission factors, and compares that total against a reduction target the household has set. It raises a flag when a monthly target is missed, plots the historical emissions trend, and lets an assigned sustainability coach review a household's history and submit written recommendations that the household can read back. Two user roles therefore share one dataset: a resident, who owns and edits the data, and a coach, who may read it and add advice but must not alter it.

2. Components Identified

Component	Layer	Responsibility
Resident Web UI	Presentation	Screens for logging consumption, setting a target, viewing the trend chart and coach notes
Coach Web UI	Presentation	Screens for reviewing assigned households and submitting recommendations
Application Facade	Business	Single entry point for the UI; authenticates every request and dispatches it to the right service
Authentication & Access Control	Business	Verifies identity and enforces resident / coach role permissions
Consumption Logging Service	Business	Validates and records electricity, gas and transport readings
Emission Calculation Engine	Business	Converts readings into monthly CO ₂ e totals using emission factors
Target & Progress Tracking Service	Business	Creates and edits reduction targets; evaluates actual emissions against the target
Alert & Notification Service	Business	Raises a flag to the household when a monthly target is missed
Reporting & Analytics Service	Business	Supplies the historical monthly series used to draw the emissions trend chart
Coaching & Recommendation Service	Business	Exposes household history to a coach; stores and serves the coach's notes
Data Access Component	Data	Single persistence gateway; the only component that issues database queries
Household Database	Data	Stores households, readings, CO ₂ e totals, targets, alerts and coach notes
Emission Factor Repository	Data	Reference data: emission factor per unit of electricity, gas and transport

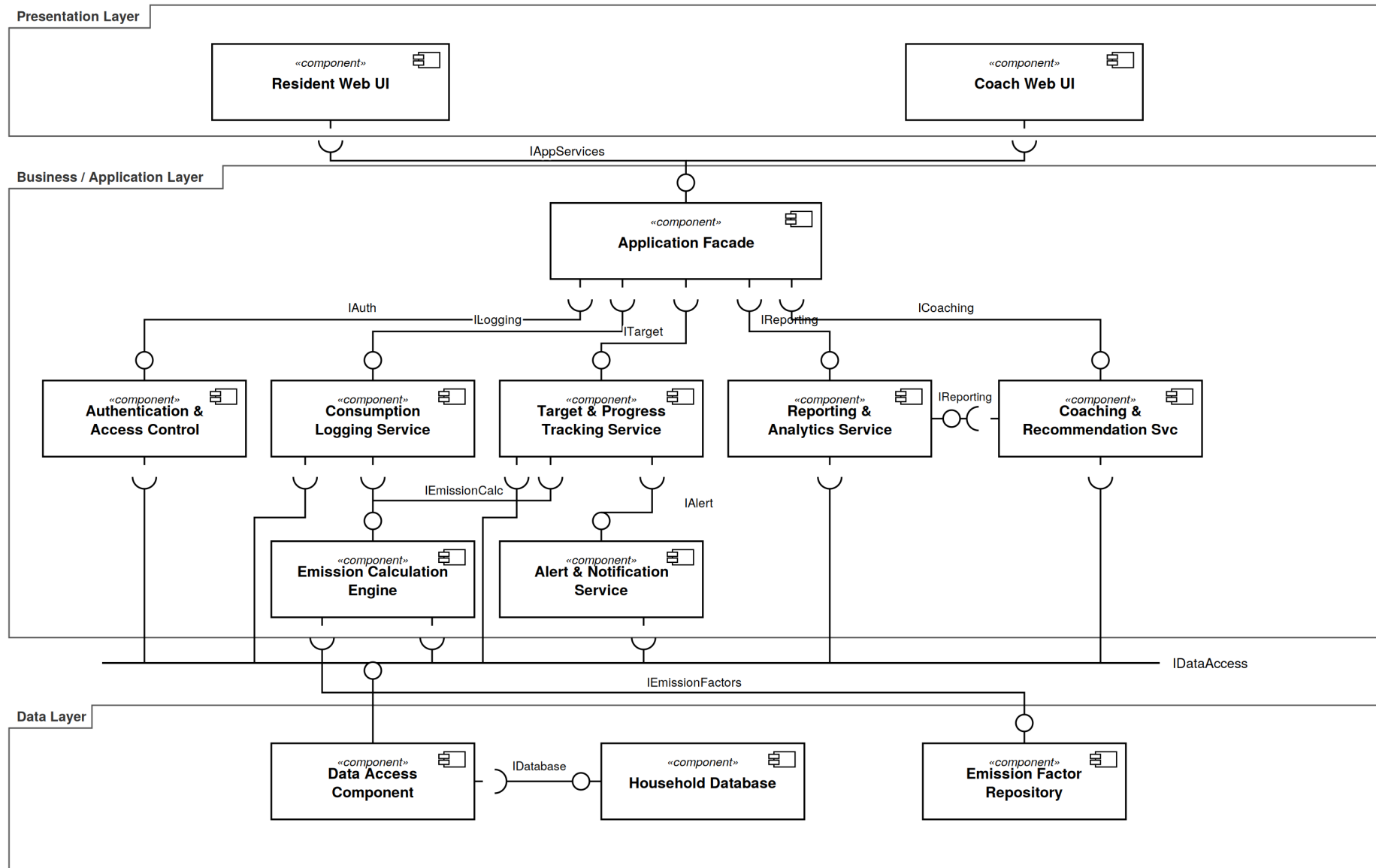
3. Interfaces

Interface	Provided by	Required by	Purpose
IAppServices	Application Facade	Resident Web UI, Coach Web UI	All user requests entering the system (HTTPS / REST)
IAuth	Authentication & Access Control	Application Facade	Login, session validation and role check on every request
ILogging	Consumption Logging Service	Application Facade	Submit an electricity, gas or transport reading
ITarget	Target & Progress Tracking Service	Application Facade	Set or edit a reduction target and read progress against it
IReporting	Reporting & Analytics Service	Application Facade, Coaching & Recommendation Service	Monthly CO ₂ e series used for the trend chart and coach review
ICoaching	Coaching & Recommendation Service	Application Facade	Submit a recommendation; read a coach's notes
IEmissionCalc	Emission Calculation Engine	Consumption Logging Service, Target & Progress Tracking Service	Convert a reading or a month of readings into CO ₂ e
IAlert	Alert & Notification Service	Target & Progress Tracking Service	Raise a missed-target alert for a household
IDataAccess	Data Access Component	All business-layer services	Persistence operations: read, insert and update domain records
IDatabase	Household Database	Data Access Component	Database connection and SQL query execution
IEmissionFactors	Emission Factor Repository	Emission Calculation Engine	Look up the emission factor for a given energy or transport type

The diagram on the following page uses the standard UML ball-and-socket notation: a circle marks an interface a component provides, and a matching arc marks an interface a component requires. The *IDataAccess* interface is drawn once as a shared connector because every business-layer service depends on it.

4. Component Diagram

Household Carbon Footprint & Target Tracker — UML Component Diagram (Layered Architecture)



○ provided interface (required interface — assembly connector

5. Architectural Justification

Architecture Selection: We chose the **Layered Architecture** for the Household Carbon Footprint & Target Tracker System, organised into a Presentation Layer, a Business / Application Layer and a Data Layer.

Reason 1 — every feature operates on one small, tightly-coupled dataset

Each capability in this system is a stage in one pipeline over the same household records. A reading is logged, converted to CO₂e, compared against that household's target, re-read to draw the trend chart, and re-read again when a coach reviews the household. Because all of it revolves around a single set of consumption records, placing one Data Layer behind the single *IDataAccess* interface keeps exactly one authoritative copy of that data and lets a target check, an alert and a chart all read consistent numbers within one transaction. A microservices split would scatter this one small dataset across separate stores and force distributed transactions and eventual-consistency handling for a problem a single database solves directly.

Reason 2 — the workload is small and uniform, so independent scaling buys nothing

A household enters a handful of readings a month and a coach reviews a handful of households. No component has a load profile different enough from the others to justify scaling or deploying it on its own, which is the principal reason to accept the operational cost of microservices. Layering still delivers the separation of concerns the project needs: updating the emission factor table touches only the Data Layer, changing how a target miss is detected touches only the Target & Progress Tracking Service, and adding a mobile client means adding one component to the Presentation Layer, because it reaches the system solely through *IAppServices*. Client-Server was rejected because it describes only the deployment topology and gives no guidance on decomposing the business logic behind the server.

Security Advantage

Layering creates a single chokepoint for access control. Both the Resident Web UI and the Coach Web UI reach the system only through *IAppServices* on the Application Facade, and the Facade calls *IAuth* on Authentication & Access Control before dispatching any request. The Presentation Layer has no path to the Data Layer — it holds no database credentials and cannot issue a query — so a tampered or compromised client cannot read another household's records directly. This matters because the two roles have deliberately different rights: a coach may read a household's emission history and post recommendations through *ICoaching*, but must never edit that household's readings or targets. Enforcing that rule at one boundary means it is written once and cannot be bypassed by calling a service directly.

Performance Benefit

Because the Emission Calculation Engine is the only component that reads the Emission Factor Repository through *IEmissionFactors*, and that reference data changes very rarely, the engine can hold the factor table in memory and convert a reading without a database round-trip each time. The monthly CO₂e totals it produces are stored once, so the Reporting & Analytics Service draws the historical trend chart from roughly twelve pre-computed rows per household per year instead of re-aggregating every reading ever logged. All of these calls are in-process calls between layers, so none of them pays the network latency and serialisation cost that the same sequence would incur if each service were deployed separately.